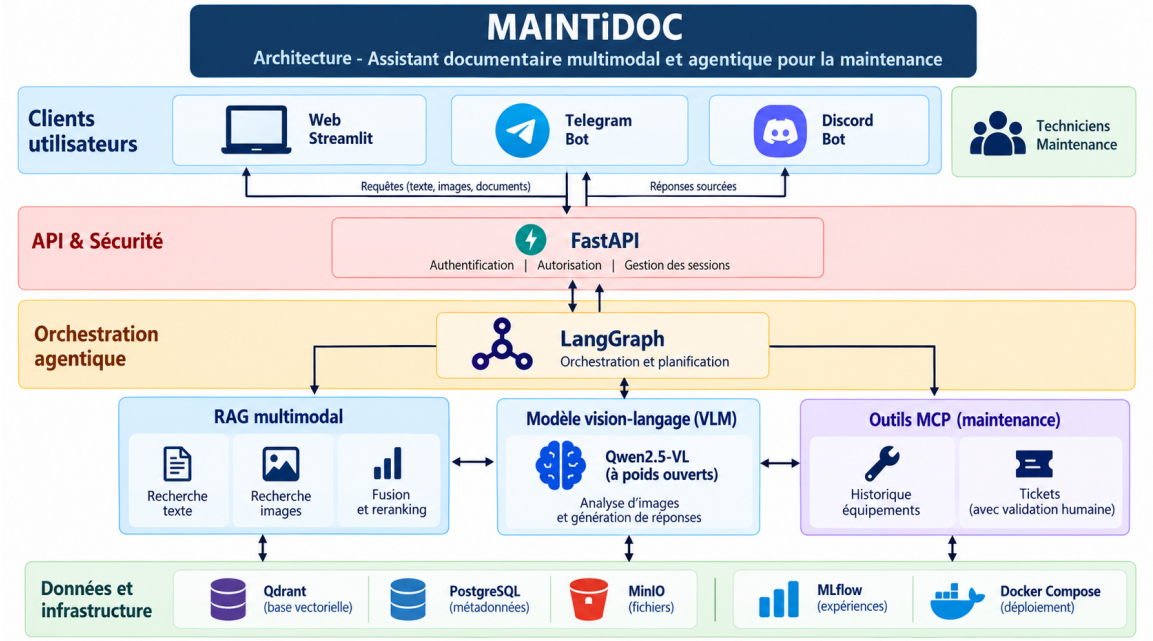
*Projet intégrateur d'IA générative multimodale et agentique*

# Feuille de route du projet

**Pr. Mohamed YOUSSFI**




## Objectif général

Construire progressivement un assistant documentaire destiné aux techniciens de maintenance, capable d'exploiter des documents et des photographies, de produire des réponses sourcées, de consulter un historique via MCP et de préparer des tickets soumis à validation humaine. Le système ne pilote aucun équipement et ne remplace pas un technicien habilité.

## Synthèse du planning

| Étape | Période | Version livrée |
|---|---|---|
| **1. Socle documentaire et RAG textuel** | Semaine 1 | V1 — Assistant textuel |
| **2. Multimodalité et fine-tuning** | Semaines 2–3 | V2 — Assistant multimodal |
| **3. Orchestration agentique et MCP** | Semaine 4 | V3 — Assistant agentique |
| **4. Bot, sécurité et déploiement** | Semaines 5–6 | V4 — Solution intégrée |

## Étape 1 — Socle documentaire et RAG textuel

*Semaine 1*

Objectif. Développer un premier assistant de questions-réponses techniques appuyé sur un corpus documentaire.

### Travaux à réaliser

- Préparer 30 à 50 documents autorisés couvrant deux familles d'équipements.
- Extraire les contenus PDF avec PyMuPDF et OCR local si nécessaire.
- Découper les documents en passages et conserver les métadonnées, versions, pages et droits d'accès.
- Générer les embeddings (BGE-M3) et indexer les passages dans Qdrant.
- Créer la chaîne RAG avec LangChain, une API FastAPI et une interface Streamlit minimale.

### Livrables

| Référence | Livrable attendu |
|---|---|
| **L1.1** | Note de cadrage et architecture |
| **L1.2** | Pipeline d'ingestion documentaire |
| **L1.3** | Index Qdrant et métadonnées |
| **L1.4** | API FastAPI et chaîne RAG |
| **L1.5** | Interface de démonstration Streamlit |
| **L1.6** | Rapport de tests documentaires et d'accès |

Critère de validation. Une question technique produit une réponse fondée sur des passages accessibles, avec références vérifiables aux documents et pages.

## Étape 2 — IA multimodale et fine-tuning

*Semaines 2–3*

Objectif. Étendre l'assistant aux photographies, schémas et figures, puis mesurer l'intérêt d'un reranker adapté.

### Travaux à réaliser

- Intégrer un VLM à poids ouverts, par exemple Qwen2.5-VL-7B-Instruct.
- Extraire et indexer les figures et schémas; réaliser la recherche texte-image avec un encodeur multilingue compatible CLIP.
- Fusionner les résultats de recherche et appliquer un reranking.
- Préparer 800 à 1 500 exemples vérifiés pour adapter un petit reranker multilingue.
- Comparer les configurations sans reranker, avec reranker initial et reranker adapté (nDCG@5, MRR@10).
- Produire des sorties structurées Pydantic, avec citations, incertitudes et demandes de clarification.

### Livrables

| Référence | Livrable attendu |
|---|---|
| **L2.1** | Module d'analyse d'images avec VLM |
| **L2.2** | Pipeline RAG multimodal |

| | |
|---|---|
| **L2.3** | Jeu de données d'adaptation |
| **L2.4** | Reranker adapté et versionné |
| **L2.5** | Rapport comparatif des métriques |
| **L2.6** | Application V2 texte + photographie |

Critère de validation. Une photographie d'équipement donne lieu à une réponse documentaire sourcée ou à une clarification explicite en cas d'ambiguïté.

## Étape 3 — Orchestration agentique et MCP

*Semaine 4*

Objectif. Intégrer les consultations d'outils et la préparation de tickets dans un workflow contrôlé.

### Travaux à réaliser

- Concevoir le graphe d'états et de transitions avec LangGraph.
- Développer un serveur MCP simulant le service de maintenance.
- Exposer get_equipment_history(equipment_id), get_open_tickets(equipment_id) et create_ticket(equipment_id, title, description, approval_token).
- Intégrer un client MCP et les contrôles d'autorisation côté serveur.
- Générer des brouillons de tickets et interrompre le workflow pour une approbation humaine explicite.
- Associer l'approbation au contenu exact et à l'utilisateur; prévenir les doublons par idempotence.

### Livrables

| Référence | Livrable attendu |
|---|---|
| **L3.1** | Workflow LangGraph documenté |
| **L3.2** | Serveur MCP de démonstration |
| **L3.3** | Client MCP intégré |
| **L3.4** | Génération de brouillons de tickets |
| **L3.5** | Validation humaine et contrôles d'accès |
| **L3.6** | Démonstration de bout en bout |

Critère de validation. L'assistant consulte l'historique, propose un ticket et ne crée celui-ci qu'après approbation explicite.

## Étape 4 — Client Web, Telegram/Discord, sécurité et déploiement

*Semaines 5–6*

Objectif. Rendre le système accessible par messagerie et fournir une solution intégrée, testée et reproductible.

### Travaux à réaliser

- Développer au choix un bot Telegram (aiogram ou python-telegram-bot) ou Discord (discord.py), relié à FastAPI.

- Prendre en charge questions, envoi de photographies, historique, tickets, approbation et annulation.
- Associer les comptes de messagerie à des identités applicatives autorisées; vérifier les permissions côté serveur.
- Déployer les services avec Docker Compose et versionner les dépendances, prompts et modèles.
- Mettre en place les tests d'intégration, d'injection de prompt, de contrôle d'accès et d'approbation.
- Évaluer environ 150 scénarios et suivre latence, qualité des citations, sécurité et ressources; documenter les limites.

**Livrables**

| Référence | Livrable attendu |
|---|---|
| **L4.1** | Bot Telegram ou Discord opérationnel |
| **L4.2** | Backend intégré avec gestion des identités |
| **L4.3** | Docker Compose et guide d'installation |
| **L4.4** | Tests automatisés et scénarios de sécurité |
| **L4.5** | Rapport d'évaluation et de performances |
| **L4.6** | Documentation utilisateur et technique |
| **L4.7** | Démonstration et soutenance finale |

Critère de validation. Depuis Telegram ou Discord, un technicien transmet une photo, reçoit une réponse sourcée, consulte un historique et approuve un ticket; les actions non autorisées sont bloquées.

## Commandes proposées pour le bot

| Commande indicative | Fonction |
|---|---|
| **/start** | Initialiser une session |
| **/ask** | Question documentaire |
| **/analyze** | Analyse d'une photographie |
| **/history** | Historique d'un équipement |
| **/tickets** | Tickets ouverts |
| **/create_ticket** | Préparer un ticket |
| **/approve** | Approuver le contenu exact |
| **/cancel** | Annuler une action en attente |

## Évaluation pédagogique proposée

| Étape | Pondération |
|---|---|
| **1 — RAG documentaire** | 20 % |
| **2 — Multimodalité et fine-tuning** | 30 % |
| **3 — Agentique et MCP** | 30 % |
| **4 — Client et déploiement** | 20 % |

À chaque jalon : code source versionné, README, procédure d'exécution, tests et démonstration. Les quatre étapes constituent des incréments successifs d'une seule application, et non quatre projets indépendants.

## Contraintes transversales

- Citations vérifiables et filtrage des documents selon les autorisations.
- Pas de pilotage de machines ni de diagnostic garanti.
- Approbation humaine obligatoire avant création de ticket.
- Secrets protégés, entrées documentaires non fiables, limites d’appels et journalisation minimale.
- Mesures expérimentales présentées comme objectifs ou résultats observés, jamais comme garanties.